

UNIX COMMANDS TUTORIAL

who

Syntax

who [options] [file]

Description

With no options, list the names of users currently logged in to the system.

Frequently used options

am i print user name of the invoking user (similar to id)
u report terminal usage. Present idle time (HOURS:MINUTES, . (dotless than one minute), or old (more than 24h).
H print headings.

pwd

Syntax

Pwd

Description

Present Working Directory (pwd). Print the full pathname of the current directory

man

Syntax

man subject

Description

Display information about a subject (name of a command).

more

Syntax

more [options] [file]

Description

Display file content on a terminal one screenful at a time. To change the next screenful use SPACEBAR and to scroll the lines use ENTER. h for help, / to search, or :n to go to the next file.

date

Syntax

date “+% option”

Description

To display dates in different formats

Frequently used options

%y yy two digit year 00-99
%Y is keyword for four digit year
%d is keyword for two digit date
%a weekday, abbreviated Wed
%A weekday, full Wednesday
%d day of the month (dd), zero padded 10
%e day of the month (dd) 10
%j day of year, zero padded 000-366
%u day of week starting with Monday (1)

%w day of week starting with Sunday (0)
%m mm month as two digits
%b Mon, locale's abbreviated Feb
%B locale's full month, variable length February
%H hour (24 hour clock) zero padded 20
%M MM minutes 28
%S SS second 00-60

passwd

Syntax

passwd [options][user]

Description

create or change a password associated with a user name. Only the owner or a privileged user may change a password.

cat

Syntax

Cat [filename][I/O redirector][filename1]

Description

Cat command is used to display the filename or append the file or combining two or more files.

Example

cat file Display the file

cat file1 file2 > files Combine file1 and file2 into files

cat file1 >> file2 Append file1 at the end of file2

clear

Syntax

clear

Description

To clear the terminal display

echo

Syntax

echo [options][string]

Description

Echo arguments to standards outputs. Often used for producing prompts from shell scripts.

Frequently used options

\a alert

\n new line

\r carriage return

\t tab

chmod

Syntax

chmod [options]mode files

Description

Change the access mode of one or more files. Only the owner or privileged user may change its mode. Create mode by concatenating *who*, *opcode* and *permission*.

Frequently used options

who (u user)(g group)(o other)(a all)
opcode (+ add permission)(remove permission)(= assign permission, and remove from the not assigned)
permission (r or 4 - read)(w or 2 - write)(x or 1 - execute)
-f quiet
-R recursive

Example

chmod u+x file (add permission to the user to execute the file)
chmod 751 file (chmod u=rwx, g=rx, o=x file) They are the same command.

chgrp

syntax

chgrp [options] newgroup *files* of chgrp.

Description

chgrp command is used to change the group of a file or directory. You must own the file or be a superuser. Newgroup is either a group id or a group name located in */etc/group* .

Frequently used Options:

- h will change the group on symbolic links.
- R recursively descend through directory changing group of all files and subdirectories.

chown

Syntax

chown [options] newowner *files*

Description

chown command to change ownership of a file or directory to one or more users.

Frequently used options

- h will change the owner on symbolic links.
- R will recursively descend through the directory, including subdirectories and symbolic links.

ls

Syntax

ls [options] [names]

Declaration

List the files in the current directory. With names is specified, list the files inside the directory names or that match a file names.

Frequently used options

- a list all files, including the hidden . files.
- d list only directory information, not its contents. Used with l.
- h human readable, using abbreviations for kilobyte, megabyte, etc.
- k size in kilobytes
- l long format list (includes permissions, owner, size, modification time, etc).
- r list files in reverse order (by name or by time).
- R recursively list subdirectories.
- s list size in blocks (1block = 512 KB)
- t list files according to modification time (newest first)
- u list files according to the file access time.

cut

Syntax

cut options [files]

Description

Cut out (that is, print) selected columns or fields from one or more *files*. The source file is not changed. This is useful if you need quick access to a vertical slice of a file. By default, the slices are delimited by a **tab**.

Frequently used options

- b *list* Print bytes in *list* positions.
- c *list* Print characters in *list* columns.
- d *delim* Set field delimiter for -f.
- f *list* Print *list* fields.

Examples

Show usernames (in the first colon-delimited field) from */etc/passwd*:

```
$ cut -d: -f1 /etc/passwd
```

Show first column of */etc/passwd*:

```
$ cut -c 1 /etc/passwd
```

paste

Syntax

paste [options] files

Description

Paste together corresponding lines of one or more *files* into vertical columns.

Frequently used options

- d'*n*' Separate columns with character *n* in place of the default **tab**.
- s Merge lines from one file into a single line. When multiple files are specified, their contents are placed on individual lines of output, one per file.

For the following three examples, *file1* contains:

```
1
2
3
```

and *file2* contains:

```
A
B
C
```

Example 1

A simple paste creates columns from each file in standard output:

```
$ paste file1 file2
```

```
1 A
2 B
3 C
```

Example 2

The column separator option yields columns separated by the specified character:

```
$ paste -d'@' file1 file2
```

```
1@A
2@B
3@C
```

Example 3

The single-line option (-s) yields a line for each file:

```
$ paste -s file1 file2
```

```
1 2 3
```

```
A B C
```

head

Syntax

```
head [options] [files]
```

Description

Print the first few lines of one or more files (the "head" of the file or files). When more than one file is specified, a header is printed at the beginning of each file, and each is listed in succession.

Frequently used options

- c *n* Print the first *n* bytes, or if *n* is followed by *k* or *m*, print the first *n* kilobytes or megabytes, respectively.
- l *n* Print the first *n* lines. The default is 10.

tail

Syntax

```
tail [options] [files]
```

Description

Print the last few lines of one or more *files* (the "tail" of the file or files). When more than one file is specified, a header is printed at the beginning of each file, and each is listed in succession.

Frequently used options

- c *n* This option prints the last *n* bytes, or if *n* is followed by *k* or *m*, the last *n* kilobytes or megabytes, respectively.
- f Follow the output dynamically as new lines are added to the bottom of a file.
- n *m* Prints the last *m* lines. The default is 10.
- f Continuously display a file as it is actively written by another process. This is useful for watching log files as the system runs.

sort

Syntax

```
sort [option] [+fields] filename [-o output file]
```

Description

Sorts the lines in a text file.

Frequently used options

-b	Ignores spaces at beginning of the line.
-d	Uses dictionary sort order and ignores the punctuation.
-f	Ignores caps
-i	Ignores nonprinting control characters.
-m	Merges two or more input files into one sorted output.
-M	Treats the first three letters in the line as a month (such as may.)
-n	Sorts by the beginning of the number at the beginning of the line.

-r	Sorts in reverse order
-u	If line is duplicated only display once
+fields	Sorts by fields , usually by tabs
filename	The name of the file that needs to be sorted.
-o outputfile	Sends the sorted output to a file.

uniq

Syntax

uniq [options] [-f fields] [-s char] [-n] [+m] [input_file [output_file]]

Declaration

Report or filter out repeated lines in a file

Frequently used options

-c	Precede each output line with a count of the number of times the line occurred in the input.
-d	Suppress the writing of lines that are not repeated in the input.
-u	Suppress the writing of lines that are repeated in the input.
-f fields	Ignore the first fields fields on each input line when doing comparisons, where fields is a positive decimal integer. A field is the maximal string matched by the basic regular expression: <code>[[:blank:]]*[^[:blank:]]*</code> If fields specifies more fields than appear on an input line, a null string will be used for comparison.
-s char	Ignore the first chars characters when doing comparisons, where chars is a positive decimal integer. If specified in conjunction with the -f option, the first chars characters after the first fields fields will be ignored. If chars specifies more characters than remain on an input line, a null string will be used for comparison.
-n	Equivalent to -f fields with fields set to n.
+m	Equivalent to -s chars with chars set to m.
input_file	A path name of the input file. If input_file is not specified, or if the input_file is -, the standard input will be used.
output_file	A path name of the output file. If output_file is not specified, the standard output will be used. The results are unspecified if the file named by output_file is the file named by input_file.

cmp

Syntax

cmp file1 file2

Declaration

Compare two files (0 identical)(1 different)(2 inaccessible)

Comm

Syntax

comm [options] file1 file2

Description

Compare file1 and file2. Outputs three columns: lines unique to file1, lines unique to file2 and lines common to both file.

Frequently used options

- 1 suppress printing column 1
- 13 suppress printing columns 1 and 3

diff

Syntax

diff [options] file1 file2

Description

Reports the differences between file1 and file2. Prints file1 text flagged (<) and file2(>).

Frequently used options

- b ignore blank spaces.
- e produce a script of commands to recreate file2 from file1, using ed editor.
- D merge file1 and file2 into a single file containing conditional C preprocessors (#ifdef). Defining symbol and then compiling yields file2. Not defining symbol, yields file1.

nl

Syntax

nl [options] [files]

Description

Number the lines of *files*, which are concatenated in the output. This command is used for numbering lines in the body of text, including special header and footer options normally excluded from the line numbering. The numbering is done for each *logical page*, which is defined as having a header, a body, and a footer. These are delimited by the special strings \:\:\:, \:\:, and \:, respectively.

Frequently used options

- b *style* Set body numbering style to *style*, t by default.
- f *style* Set footer number style to *style*, n by default.
- h *style* Set header numbering style to *style*, n by default.

Styles can be in these forms:

- A Number all lines.
- t Only number non-empty lines.
- n Do not number lines.
- p *REGEXP* Only number lines that contain a match for regular expression *REGEXP*.

Example

Suppose file *file1* contains the following text:

\:\:\:

header

\:\:

line1

```
line2
line3
\:\:
header
\:\:
line1
line2
line3
\:\:
header
```

If the following command is given:

```
$ nl -h a file1
```

the output would yield numbered headers and body lines but no numbering on footer lines. Each new header represents the beginning of a new logical page and thus a restart of the numbering sequence:

```
1 header
2 line1
3 line2
4 line3
header
1 header
2 line1
3 line2
4 line3
Footer
```

pr

Syntax

```
pr [options] [file]
```

Description

Convert a text *file* into a paginated, columnar version, with headers and page fills. This command is convenient for yielding nice output, such as for a line printer from raw uninteresting text files. The header will consist of the date and time, the filename, and a page number.

Frequently used options

-d	Double space.
-h header	Use <i>header</i> in place of the filename in the header.
-l lines	Set page length to <i>lines</i> . The default is 66.
-o width	Set the left margin to <i>width</i> .

split

Syntax

```
split [option] [infile] [outfile]
```

Description

Split *infile* into a specified number of line groups, with output going into a succession of files, *outfileaa*, *outfileab*, and so on (the default is *xaa*, *xab*, etc.). The *infile* remains unchanged. This command is handy if

you have a very long text file that needs to be reduced to a succession of smaller files. This was often done to email large files in smaller chunks, because it was at one time considered bad practice to send single large email messages.

Frequently used option

-n Split the *infile* into *n*-line segments. The default is 1000.

Example

Suppose *file1* contains:

```
1 one
2 two
3 three
4 four
5 five
6 six
```

Then the command:

```
$ split -2 file1 splitout_
```

yields as output three new files, *splitout_aa*, *splitout_ab*, and *splitout_ac*. The file *splitout_aa* contains:

```
1 one
```

```
2 two
```

splitout_ab contains:

```
3 three
```

```
4 four
```

and *splitout_ac* contains:

```
5 five
```

```
6 six
```

tr

Syntax

```
tr [options] [[string1 [string2]]
```

Description

Translate characters from *string1* to the corresponding characters in *string2*. *tr* does *not* have file arguments and therefore must use standard input and output. If *string1* and *string2* specify ranges (a-z or A-Z), they should represent the same number of characters.

Frequently used options

-d Delete characters in *string1* from the output.

-s Squeeze out repeated output characters in *string1*.

Example 1

To change all lowercase characters in *file1* to uppercase, use either of these commands:

```
$ cat file1 | tr a-z A-Z
```

or:

```
$ tr a-z A-Z < file1
```

Example 2

To suppress repeated "a" characters from *file1*:

```
$ cat file1 | tr -s a
```

Example 3

To remove all "a," "b," and "c" characters from *file1*:

```
$ cat file1 | tr -d abc
```

wc

Syntax

wc [options] [files]

Description

Print counts of characters, words, and lines for *files*. When multiple files are listed, statistics for each file output on a separate line with a cumulative total output last.

Frequently used options

- c Print the character count only.
- l Print the line count only.
- w Print the word count only.

Example 1

Show all counts and totals for *file1*, *file2*, and *file3*:

```
$ wc file[123]
```

Example 2

Count the number of lines in *file1*:

```
$ wc -l file1
```

cp

Syntax

cp [options] file1 file2

cp [options] files directory

Description

In the first command form, copy *file1* to *file2*. If *file2* exists and you have appropriate privileges, it will be overwritten without warning (unless you use the *-i* option). Both *file1* and *file2* can be any valid filename, either fully qualified or in the local directory. In the second command form, copy one or more *files* to *directory*. Note that the presence of multiple files *implies* that you wish to copy files to a directory. If *directory* doesn't exist, an error message will be printed. This command form can get you in trouble if you attempt to copy a single file into a directory that doesn't exist, as the command will be interpreted as the first form and you'll end up with *file2* instead of *directory*.

Frequently used options

- f Force an overwrite of existing files in the destination.
- i Prompt *interactively* before overwriting destination files. It is common practice (and advised) to alias the **cp** command to **cp -i** to prevent accidental overwrites. You may find that this is already done for you for user root on your Linux system.
- p Preserve all information, including owner, group, permissions, and timestamps. Without this option, the copied file or files will have the present date and time, default permissions, owner, and group.
- r, -R Recursively copy directories. You may use either upper- or lowercase for this option. If *file1* is actually a directory instead of a file and the recursive option is specified, *file2* will be a copy of the entire hierarchy under directory *file1*.
- v Display the name of each file verbosely before copying.

Example 1

Copy the messages file to the local directory (specified by *.*):

```
$ cp /var/log/messages .
```

Example 2

Make an identical copy, including preservation of file attributes, of directory *src* in new directory *src2*:

```
$ cp -Rp src src2
```

Copy *file1*, *file2*, *file5*, *file6*, and *file7* from the local directory into your home directory (under *bash*):

```
$ cp file1 file2 file[567] ~
```

grep

Syntax

```
grep [options] regex [files]
```

Description

Search *files* or standard input for lines containing a match to regular expression *regex*. By default, matching lines will be displayed and nonmatching lines will not be displayed. When multiple files are specified, **grep** displays the filename as a prefix to the output lines (use the **-h** option to suppress filename prefixes).

Frequently used options

- c** Display only a count of matched lines, but not the lines themselves.
- h** Display matched lines, but do not include filenames for multiple file input.
- i** Ignore uppercase and lowercase distinctions, allowing *abc* to match both *abc* and *ABC*.
- n** Display matched lines prefixed with their line numbers. When used with multiple files, *both* the filename and line number are prefixed.
- v** Print all lines that *do not* match *regex*. This is an important and useful option. You'll want to use regular expressions, not only to *select* information but also to *eliminate* information. Using **-v** inverts the output this way.

Examples

Since regular expressions can contain both metacharacters and literals, **grep** can be used with an entirely literal *regex*. For example, to find all lines in *file1* that contain either *Linux* or *linux*, you could use **grep** like this:

```
$ grep -i linux file1
```

In this example, the *regex* is simply "linux." The uppercase *L* in "Linux" is matched by the command-line option **-i**. This is fine for literal expressions that are common.

However, in situations in which *regex* includes regular expression metacharacters that are also shell special characters (such as **\$** or *****), the *regex* must be quoted to prevent shell expansion and pass the metacharacters on to **grep**.

As a simplistic example of this, suppose you have files in your local directory named *abc*, *abc1*, and *abc2*. When combined with **bash**'s **echo** expression, the *abc** wildcard expression lists all files that begin with *abc*, as follows:

```
$ echo abc*
abc abc1 abc2
```

Now suppose that these files contain lines with the strings *abc*, *abcc*, *abccc*, and so on, and you wish to use **grep** to find them. You can use the shell wildcard expression *abc** to expand to all the *abc* files as displayed with **echo** above, and you'd use an identical regular expression *abc** to find all occurrences of lines containing *abc*, *abcc*, *abccc*, etc.

Without using quotes to prevent shell expansion, the command would be:

```
$ grep abc* abc*
```

After shell expansion, this yields:

```
$ grep abc abc1 abc2 abc abc1 abc2 # no!
```

This is *not* what you intended! **grep** would search for the literal expression *abc*, because it appears as the first command argument. Instead, quote the regular expression with single or double quotes to protect it:

```
$ grep 'abc*' abc*
```

or:

```
$ grep "abc*" abc*
```

After expansion, both examples yield the same results:

egrep

Syntax

```
egrep [options] [regex] [files]
```

Description

Search files that match a regular expression [regex]. Support metacharacters +, ?, (), etc. But must be enclosed with quotes.

Frequently used options

-n print the line matching.

-v print lines that don't match [regex].

Example

Example: `egrep '(Victor | Victoria)' file`

fgrep

Syntax

```
fgrep [options][pattern] [files]
```

Description

Fast grep. Search files for lines that match a literal. Options are the same as for egrep.

mkdir

Syntax

```
mkdir [options] directories
```

Description

Create one or more *directories*. You must have write permission in the directory where *directories* are to be created.

Frequently used options

-m *mode* Set the access *mode* for *directories*.

-p Create intervening parent directories if they don't exist.

Examples

Create a read-only directory named *personal*:

```
$ mkdir -m 444 personal
```

Create a directory tree in your home directory, as indicated with a leading tilde (~), using a single command:

```
$ mkdir -p ~/dir1/dir2/dir3
```

In this case, all three directories are created. This is faster than creating each directory individually.

mv

Syntax

```
mv [options] source target
```

Description

Move or rename files and directories. For *targets* on the same filesystem (partition), moving a file doesn't relocate the contents of the file itself. Rather, the directory entry for the target is updated with the new

location. For *targets* on different filesystems, such a change can't be made, so files are copied to the target location and the original sources are deleted.

Note that **mv** is used to rename files and directories, because a rename operation requires the same directory entry update as a move.

If a *target* file or directory does not exist, *source* is renamed to *target*. If a *target* file already exists, it is overwritten with *source*. If *target* is an existing directory, *source* is moved into that directory. If *source* is one or more files and *target* is a directory, the files are moved into the directory.

Frequently used options

- f Force the move even if *target* exists, suppressing warning messages.
- i Query interactively before moving files.

rm

Syntax

rm [options] files

Description

Delete one or more *files* from the filesystem. To remove a file, you must have write permission in the directory that contains the file, but you do not need write permission on the file itself. The **rm** command also removes directories when the **-d**, **-r**, or **-R** option is used.

Frequently used options

- d Remove directories even if they are not empty. This option is reserved for privileged users.
- f Force removal of write-protected files without prompting.
- i Query interactively before removing files.
- r, -R If the *file* is a directory, recursively remove the entire directory and all of its contents, including subdirectories.

rmdir

Syntax

rmdir [option] directories

Description

Delete *directories*, which must be empty.

Frequently used option

- p Remove *directories* and any intervening parent directories that become empty as a result. This is useful for removing subdirectory trees.

Remember that recursive remove using **rm -R** removes directories too, even if they're not empty.

touch

Syntax

touch [options] files

Description

Change the access and/or modification times of *files*. This command is used to refresh timestamps on files. Doing so may be necessary, for example, to cause a program to be recompiled using the date-dependant **make** utility.

Frequently used options

- a Change only the access time.

- m** Change only the modification time.
- t *timestamp*** Instead of the current time, use *timestamp* in the form of `[[CC]YY]MMDDhhmm[.ss]`. For example, the *timestamp* for January 12, 2001, at 6:45 p.m. is 200101121845.

tee

Syntax

`tee [options] files`

Description

Read from standard input and write both to one or more *files* and to standard output (analogous to a tee junction in a pipe).

Option

- a** Append to *files* rather than overwriting them.

Example

Suppose you're running a pipeline of commands `cmd1`, `cmd2`, and `cmd3`:

```
$ cmd1 | cmd2 | cmd3 > file1
```

This sequence puts the ultimate output of the pipeline into *file1*. However, you may also be interested in the intermediate result of `cmd1`. To create a new *file_cmd1* containing those results, use **tee**:

```
$ cmd1 | tee file_cmd1 | cmd2 | cmd3 > file1
```

The results in *file1* will be the same as in the original example, and the intermediate results of `cmd1` will be placed in *file_cmd1*.

ps

Syntax

`ps [options]`

Description

This command generates a one-time snapshot of the current processes on standard output.

Frequently used options

- a** Show processes that are owned by other users and attached to a terminal. Normally, only the current user's processes are shown.
- f** "Forest" mode, which is used to display process family trees. For example, using this option will display all running child web servers (`httpd`) in a hierarchical diagram under the parent web server.
- l** Long format, which includes priority, parent PID, and other information.
- u** User format, which includes usernames and the start time of processes.
- w** Wide output format, used to eliminate the default output line truncation. Useful for the **-f** option.
- x** Include processes without controlling terminals. Often needed to see daemon processes and others not started from a terminal session.
- C *cmd*** Display instances of command name *cmd*.
- U *usr*** Display processes owned by username *usr*.

Examples

Simply entering the `ps` command with no options will yield a brief list of processes owned by you and attached to your terminal:

```
$ ps
```

Use the **-a**, **-u**, and **-x** options to include processes owned by others and not attached to terminals as well as to display them in the "user" mode. The command is valid with or without the dash:

```
$ ps -aux
```

\$ ps aux

In this case, the dash is optional. However, certain **ps** options require the dash. If you are interested in finding process information on a particular command, use the **-C** option. This command displays all web server processes:

\$ ps u -C httpd

You'll note that the **-C** option *requires* the dash, but the **u** option won't work with it if a dash is included. This confusion exists because the **ps** command as implemented on Linux understands options in three differing forms:

Unix98 options

These may be grouped and must be preceded by a dash.

BSD options

These may be grouped and must *not* be used with a dash.

GNU long options

These options are preceded by two dashes.

All of these option types may be freely intermixed. Instead of the **-C** option, you may wish to use **ps** with other options that you usually use and pipe the output to **grep**, searching for process names, PIDs, or anything else you know about the process: **\$ ps -aux | grep httpd**

In this case, the result would be the same list of **httpd** servers, as well as the **grep** command itself.

kill

Syntax

kill [-s sigspec | -sigspec] [pids]

kill -l [signum]

Description

In the first form, **kill** is used with an optional *sigspec*. This is a signal value, specified as either an integer or the signal name (such as **SIGHUP**, or simply **HUP**). The *sigspec* is case-insensitive but usually specified with uppercase letters. You may use **-s sigspec** or simply **-sigspec** to make up the signal value or name. If a *sigspec* is not given, then **SIGTERM** (signal 15, "exit gracefully") is assumed. The *sigspec* is followed by one or more *pids* to which the signal is to be sent. In the second form with the **-l** option, **kill** lists the valid signal names. If *signum* (an integer) is present, only the signal name for that number will be displayed.

Examples

This command displays the signal name **SIGTERM**, the name of signal 15, and the default when **kill** is used to signal processes:

\$ kill -l 15

All of these commands will send a **SIGTERM** signal to the processes with PIDs 1000 and 1001:

\$ kill 1000 1001

\$ kill -15 1000 1001

\$ kill -SIGTERM 1000 1001

\$ kill -sigterm 1000 1001

\$ kill -TERM 1000 1001

\$ kill -s 15 1000 1001

\$ kill -s SIGTERM 1000 1001

If those two processes are playing nicely on your system, they'll comply with the **SIGTERM** signal and terminate when they're ready (after they clean up whatever they're doing). Not all processes will comply, however. A process may be hung in such a way that it cannot respond, or it may have *signal handling* code written to trap the signal you're trying to send. To force a process to die, use the strongest **kill**:

```
$ kill -9 1000 1001
$ kill -KILL 1000 1001
```

These equivalent commands send the **KILL** signal to the process, which the process cannot ignore. The process will terminate immediately without regard to closing files or other cleanup procedures. Because of this, using the **KILL** signal is a last resort.

jobs

Syntax

jobs [options]

Description

Lists the jobs that you are running in the background and in the foreground. If the prompt is returned with no information no jobs are present. Note: not all shells are capable of running this command.

Frequently used options

- p | -l Report the process group ID and working directory of the jobs.
 - n Display only jobs that have stopped or exited since last notified.
 - p Displays only the process IDs for the process group leaders of the selected jobs.
 - x Replace any job_id found in command or arguments with the corresponding process group ID, and then execute command passing it arguments.
- job id The job id.

at

Schedules a command to be ran at a particular time, such as a print job late at night.

Syntax

at	executes commands at a specified time.
atq	lists the user's pending jobs, unless the user is the superuser; in that case, everybody's jobs are listed. The format of the output lines (one for each job) is: Job number, date, hour, job class.
atrm	deletes jobs, identified by their job number.
batch	executes commands when system load levels permit; in other words, when the load average drops below 1.5, or the value specified in the invocation of atrun.

at [-c | -k | -s] [-f filename] [-q queue name] [-m] -t time [date] [-l] [-r]

Frequently used options

-c	C shell. csh(1) is used to execute the at-job.
-k	Korn shell. ksh(1) is used to execute the at-job.
-s	Bourne shell. sh(1) is used to execute the at-job.
-f filename	Specifies the file that contains the command to run.
-m	Sends mail once the command has been run.

-t time	Specifies at what time you want the command to be ran. Format hh:mm. am / pm indication can also follow the time otherwise a 24-hour clock is used. A timezone name of GMT, UCT or ZULU (case insensitive) can follow to specify that the time is in Coordinated Universal Time. Other timezones can be specified using the TZ environment variable. The below quick times can also be entered: midnight - Indicates the time 12:00 am (00:00). noon - Indicates the time 12:00 pm. now - Indicates the current day and time. Invoking at - now will submit submit an at-job for potentially immediate execution.
date	Specifies the date you wish it to be ran on. Format month, date, year. The following quick days can also be entered: today - Indicates the current day. tomorrow - Indicates the day following the current day.
-l	Lists the commands that have been set to run.
-r	Cancels the command that you have set in the past.

fg

Syntax

fg [%job]

Description

Continues a stopped job by running it in the foreground, some shells are not able to run this command.

bg

Syntax

bg [-l] [-p] [-x] [job]

Description

Continues a stopped job in the background. This function is not available on all Unix shell's.

Frequently used options

-l	Report the process group ID and working directory of the jobs.
-p	Report only the process group ID of the jobs.
-x	Replace any job_id found in command or arguments with the corresponding process group ID, and then execute command passing it arguments.
job	Specifies the job that you want to run in the background.

nice

Syntax

nice [-n adjustment] [command]

`nice [-adjustment] [command]`

Description

The `nice` command is used to alter another command's nice number at start time. For normal users, *adjustment* is an integer from 1 to 19. If you're the superuser, the *adjustment* range is from -20 to 19. If an *adjustment* number is not specified, the process' `nice` number defaults to 10. The *command* consists of any command that you might enter on the command line, including all options, arguments, redirections, and the background character `&`.

If both *adjustment* and *command* are omitted, `nice` displays the current scheduling priority, which is inherited.

Example 1

The following command starts a program in the background with reduced priority, using the default nice number of 10:

```
$ nice somecmd -opt1 -opt2 arg1 arg2 &
```

Example 2

As superuser, you can start programs with elevated priority. These equivalent commands start the `vi` editor with a higher priority, which may be necessary for administrative purposes if the system is exceptionally slow:

```
# nice --10 vi /etc/hosts.deny
```

```
# nice -n -10 vi /etc/hosts.deny
```

Note the double dash (`-- 10`) in the first form. The first dash indicates that an option follows, while the second dash indicates a negative number.

Be careful when using `nice` on interactive programs such as editors, word processors, or browsers. Assigning a program a positive nice number will most likely result in sluggish performance. Remember, the higher the positive number, the lower the resulting priority level. For that reason, you should try not to assign positive nice numbers to foreground jobs on your terminal. If the system gets busy, your terminal could hang awaiting CPU time, which has been sacrificed by `nice`.

write

Syntax

`write person [ttyname]`

Person	If you wish to talk to someone on your own machine, then <i>person</i> is just the person's login name. If you wish to talk to a user on another host, then <i>person</i> is of the form 'user@host'.
Ttyname	If you wish to talk to a user who is logged in more than once, the <i>ttyname</i> argument may be used to indicate the appropriate terminal name, where <i>ttyname</i> is of the form 'ttyXX' or 'pts/X'.

Description

Send a message to another user.

Examples

`write hope`

Typing the above command would start a talking session with the user `hope`. If you type your name, you can echo back

msg

Syntax

mesg [n|y]

N	Prevents the messages from others popping up on your screen.
Y	Allows messages to appear on your screen.

Description

Lets you control if people can use the write command, to send text to you over the screen.

Examples

mesg

Typing this command by itself will display whether or not the mesg service is enabled. For example, it may display: "is y", indicating that it is enabled. Typing n or why after the command would enable or disable this command.

When this command is disabled non-root users will be unable to to send messages to you using message commands such as talk or write.

talk

Syntax

talk person [ttyname]

person	If you wish to talk to someone on your own machine, then person is just the person's login name. If you wish to talk to a user on another host, then person is of the form 'user@host'.
ttyname	If you wish to talk to a user who is logged in more than once, the ttyname argument may be used to indicate the appropriate terminal name, where ttyname is of the form 'ttyXX' or 'pts/X'.

Description

Talk with other logged in users.

Examples

talk hope

Typing the above command would start a talking session with the user hope.

wall

Syntax

wall [message]

Description

Wall sends a message to everybody logged in with their mesg(1) permission set to **yes**. The message can be given as an argument to *wall*, or it can be sent to *wall*'s standard input. When using the standard input from a terminal, the message should be terminated with the **EOF** key (usually Control-D). The length of the message is limited to 20 lines.

mail

Syntax

mail

Description

mail command is used to send and receive mails locally and globally.

Frequently used option

- s Specify subject on command line.
- c Send carbon copies to list of users.
- b Send blind carbon copies to list. List should be a comma-separated list of names.
- f Read in the contents of your mbox for processing; when you quit, mail writes undeleted messages back to this file.
- i Ignore tty interrupt signals. This is particularly useful when using mail on noisy phone lines.

Common File-Naming Wildcards

Wildcard	Description
*	Commonly thought to "match anything." It actually will match zero or more characters (which includes "nothing"!). For example, <code>x*</code> matches files or directories <code>x</code> , <code>xy</code> , <code>xyz</code> , <code>x.txt</code> , <code>xy.txt</code> , <code>xyz.c</code> , and so on.
?	Match exactly one character. For example, <code>x?</code> matches files or directories <code>xx</code> , <code>xy</code> , <code>xz</code> , but not <code>x</code> and not <code>xyz</code> . The specification <code>x??</code> matches <code>xyz</code> , but not <code>x</code> and <code>xy</code> .
[characters]	Match any single character from among <i>characters</i> listed between the brackets. For example, <code>x[yz]</code> matches <code>xy</code> and <code>xz</code> .
[!characters]	Match any single character other than <i>characters</i> listed between the brackets. For example, <code>x[!yz]</code> matches <code>xa</code> and <code>x1</code> but does not match <code>xy</code> and does not match <code>xz</code> .
[a-z]	Match any single character from among the range of characters listed between the brackets and indicated by the dash (the dash character is not matched). For example, <code>x[0-9]</code> matches <code>x0</code> and <code>x1</code> , but does not match <code>xx</code> . Note that to match both upper- and lowercase letters,[13] you specify <code>[a-zA-Z]</code> . Using <code>x[a-zA-Z]</code> matches <code>xa</code> and <code>xA</code> .
[!a-z]	Match any single character from among the characters not in the range listed between the brackets.
{ frag1, frag2, frag3...}	Create strings <i>frag1</i> , <i>frag2</i> , <i>frag3</i> , etc. For example, <code>file_{one,two,three}</code> yields the strings <code>file_one</code> , <code>file_two</code> , and <code>file_three</code> . This is a special operator named <i>brace expansion</i> that can be used to match filenames but isn't specifically a file wildcardoperator and does not examine directories for existing files to match. Instead, it will expand <i>any string</i> . For example, it can be used with <code>echo</code> to yield strings totally unrelated to existing filenames: <pre>\$ echo string_{a,b,c} string_a string_b string_c</pre>

Standard I/O Redirections for the bash shell

Redirection Function

Send *stdout* to *file*.

Syntax for bash

`$ cmd > file`

`$ cmd 1> file`

Send *stderr* to *file*.

`$ cmd 2> file`

Send both *stdout* and *stderr* to *file*.
 Send *stdout* to *file1* and *stderr* to *file2*.
 Receive *stdin* from *file*.
 Append *stdout* to *file*.

Append *stderr* to *file*.
 Append both *stdout* and *stderr* to *file*.
 Pipe *stdout* from *cmd1* to *cmd2*.
 Pipe *stdout* and *stderr* from *cmd1* to *cmd2*.

```
$ cmd > file 2>&1
$ cmd > file1 2> file2
$ cmd < file
$ cmd >> file
$ cmd 1>> file
$ cmd 2>> file
$ cmd >> file 2>&1
$ cmd1 | cmd2
$ cmd1 2>&1 | cmd2
```

There are three types of quotes

Quotes	Name	Meaning
" "	Double Quotes	"Double Quotes" - Anything enclosed in double quotes removed meaning of that characters (except \ and \$).
' '	Single quotes	'Single quotes' - Enclosed in single quotes remains unchanged.
` `	Back quote	`Back quote` - To execute command

Wild card /Shorthand	Meaning	Examples	
*	Matches any string or group of characters.	\$ ls *	will show all files
		\$ ls a*	will show all files whose first name is starting with letter 'a'
		\$ ls *.c	will show all files having extension .c
		\$ ls ut*.c	will show all files having extension .c but file name must begin with 'ut'.
?	Matches any single character	\$ ls ?	will show all files whose names are 1 character long
		\$ ls fo?	will show all files whose names are 3 character long and file name begin with fo
[...]	Matches any one of the enclosed characters	\$ ls [abc]*	will show all files beginning with letters a,b,c